



- No Results Found for "crossover"
- [TCP API](#)
- [Device Information](#)
- [Device control](#)
- [Playback Control](#)
- [Playback Information](#)
- [Passthrough Function](#)
 - [Extended EQ function](#)
 - [AP8064 platform](#)
 - [BP10XX platform](#)

TCP API

Welcome to Arylic's TCP API developer documentation.

The Query and Control Interface is supported by standard TCP/IP Sockets. So unique socket identification on the LAN/WLAN is the pair of the Arylic Device IP Address and the Communication Port. The creation and management of TCP/IP Sockets is no covered in the Scope of this document as this can found in many places on the Internet and in books.

It is a strong recommendation to familise yourself with the Arylic "Web Management Interface". This will enable you to configure the IP address of your Arylic Device and also configure the communications Port for the TCP/IP Socket if required.

Arylic provide a simple development Tool for Windows. You can evaluate these commands without any coding.

The API is working in bi-directional, which means device would send message to TCP client when its status changed. For example, user adjust volume on APP or with remote. The TCP client would receive the volume change message.

Connection

The client could connect to device via `TCP` method on port `8899`, and the connection could be remained for sending or receiving data in the future.

i Please send the commands with interval for at least 200ms.

i Limited one connection for single IP at the same time.

Data format

The connecton are bi-directional, so client can send data to device and also will receive data from device. The actual payload are packed into a structured packet in the following format. And normally we define the payload send to device as `command` and payload received from device as `message`. The `command` all will begin with `MCU` and the `message` normally begin with `AXX` (except the `PAS` function), and then follow with function and param. The function and param are normally a 3 characters text, and add together with `+` mark, so the payload normally has 11 bytes length. And if the payload is longer than 11 bytes, you need to add an extra `&` mark in the end. The command and message will have same function name, you need to remember the difference.

For example, when you send `MCU+VOL+050` to device, that means set current system volume to 50%, and when you received `AXX+VOL+050` and that means current device have volume 50%.

sample packet with payload MCU+VOL+050

```
0x18 0x96 0x18 0x20
0x0b 0x00 0x00 0x00
0xc1 0x02 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x4d 0x43 0x55 0x2b 0x56 0x4f 0x4c 0x2b 0x30 0x35 0x30
```

Packet structure

Name	Length	Description
header	4	<code>0x18 0x96 0x18 0x20</code>
length	4	Length of the payload, little endian INT. eg: <code>0x0b 0x00 0x00 0x00</code> means 11 bytes payload.
checksum	4	Sum of all payload bytes, little endian INT.
reserved	8	<code>0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00</code>
payload	N	Command to device or message from device

Payload

set system volume to 50

MCU+VOL+050

response message for system volume

AXX+VOL+050

The commands sent to device would start with `MCU+`, and then following 3 bytes major type, and then a mark `+` with a param or minor type or action, anyway. It would be `-` in some commands.

The message received from device would start with `AXX+`, and then following 3 bytes major type, and then mark `+`, and then param or response.

Normally the length of payload would be fixed 11 bytes. For some commands/messages which is longer than 11 bytes, it supposed to be ended with mark `&`.

i In the following document, will only describe the payload, you need to package it for full command data before sending over TCP.

Device Information

Basic Information

get device information

MCU+DEV+GET

response message `AXX+DEV+INFSoundSystem_D1C2;release;SoundSystem_D1C2;52414B4F49545F52445F322E34;-36;0;0&`

The response of device information are seperated with `;`, and the description of each field:

Field in Sample	Description
SoundSystem_D1C2	device name
release	firmware build type
SoundSystem_D1C2	device SSID which would be seen in hotspot or when setting up network
52414B4F49545F52445F322E34	SSID of connected AP hexed string
-36	RSSI of current AP
0	unkown
0	unkown

More Information

get device information

MCU+INF+GET

response message

`AXX+INF+INF{ "uuid": "FF31F09E8AFCEBECAF8BF24F", "DeviceName": "SoundSystem_D1C2", "GroupName": "SoundSystem_D1C2", "ssid": "SoundSystem_D1C2", "language": "en", "curpos": "180157", "totlen": "272000", "status": "play", "loop": "0"}&`

The response will contain most of the device information, and is same with the HTTPAPI getStatusEx.

Network State

get current internet state

MCU+WWW+GET

response message

AXX+WWW+001

To query current internet state of device. And when no internet connction, it would return 000, otherwise 001.

Device will send actively when status changed.

USB Disk

get status of usb device

MCU+USB+GET

response message

AXX+USB+000

When no usb disk detected, it will return 000, and when have detected usb disk plugin, will return with 001.

Device will send actively when status changed.

Device control

Volume

get current volume

MCU+VOL+030

response message

AXX+VOL+030

The param of MUT is an integer range from 0 to 100, for example, 030 is for volume 50.

You could send MCU+VOL+GET to get current system volume.

Mute

mute

```
MCU+MUT+001
```

response message

```
AXX+MUT+001
```

The param of `MUT` is either `000` or `001`, which is unmute or mute.

You could send `MCU+MUT+GET` to check current state.

Device Name

set device name to apple

```
MCU+NAM+SETapple&
```

response message

```
AXX+NAM+SETapple&
```

When device name changed, you should see this changes on APP or else. And also when you change name on APP, the connected TCP client should also receive this change.

Device Reboot

reboot

```
MCU+DEV+RST&
```

This command is used to reboot the WIFI module only. And the client connection would be lost.

Reset Factory Settings

reset factory settings

```
MCU+FACTORY
```

This will reset device to factory. And the client connection would be lost.

Playback Control

Playback Control

pause current playback

MCU+PLY-PUS

response message

AXX+PLY+000

Please note that the mark between type and param are (-). And considering the different mark would affect the command, have added it with the param in the following table.

The responded messages would just stand for the playback status, and might have no connection with the command sent.

Param	Description
-PUS	pause current playback
+PUS	toggle play and pause
-PLA	resume play, available only when paused
-STP	stop current playback
+NXT	next track, available only when play list
+PRV	previous track, available only when play list
+PUQ	start to play last playlist, available only when ever started playback on APP

Playback Mode

set repeat one

MCU+PLP+001

response message

AXX+PLP+001

The playback mode is defined as following:

Param	Description
000	repeat all

Param	Description
001	repeat one
002	repeat all and shuffle
003	shuffle
004	sequence

You could send `MCU+PLP+GET` to query current mode.

Preset

play preset #1

```
MCU+KEY+001
```

The device support 10 groups of presets, stands for the relative group shown on APP.

Param	Description
nnn	a number from 1 to 10, to play the preset playlist
NXT	play next preset playlist
PRE	play previous preset playlist

set current playlist to preset #2

```
MCU+PRE+002
```

response message

```
AXX+PRE+FF2
```

This command will have effect only if the current source support preset. Some known working playlist: Spotify Connect, Internet Radio, TIDAL, QOBUZ.

Playback Information

Current Input Mode

message from device when switch to LINE-IN

```
AXX+MEA+RDY  
AXX+PLM+040  
AXX+VOL+030
```

The param of **PLM** is the current playback source, and you could send **MCU+PLM+GET** to query current input. And response **MEA+RDY** means the media info is ready.

Param	Description
000	Idle
001	AirPlay
002	DLNA
010	online playlist
011	USB playlist
020	HTTP API
031	Spotify Connect
032	TIDAL Connect (A97 only)
040	LINE-IN
041	Bluetooth
045	Coaxial
047	LINE-IN 2
049	HDMI
051	USB DAC
053	External Bluetooth
054	Phono
056	Optical 2
057	Coaxial 2
058	ARC
099	Slave mode

Media Information

For media information, the base board will keep query the state. So the TCP client could just monitor the relative messages. And the MCU will only query in period, so client could try to send the query command to update immediately for the current status, and then monitor messages to update.

Another point need to be noted, the information might be not accurate, you need to test with the real condition to determine which one is more suitable.

SHORT

check current info

MCU+SONGGET

response message

AXX+SNG+INF{"curpos": "3996", "totlen": "229000", "status": "play", "loop": "0"}&

This message is short, so normally used to query current progress.

Field	Description
curpos	milliseconds of elapsed
totlen	milliseconds of duration
status	current playback status, might be play , stop , pause
loop	loop/shuffle mode, please refer to the above Playback Mode

MEDIA INFORMATION

get media information

MCU+MEA+GET

response message

AXX+MEA+DAT{ "title": "4865616C2054686520576F726C642E6D7033", "artist": "4D69636861656C204A61636B736F6E", "album": "4B696E67204F6620506F70", "vendor": "5

This message will contain more information of current playback. The field title/artist/album should be comprehensive, and will list some known vendor below, the skiplimit is unknown. All messages are with format hexed string.

Vendor	Description
QPlay	QPlay
Spotify	Spotify Connect
Airplay	AirPlay
UPnPServer	playlist in UPnP Share
Tidal	TIDAL
Rhapsody	Napster
Qobuz	QOBUZ
DEEZER	DEEZER
TUNEIN	Tune In
iheart	iHeart Radio
USBQueue	USB Disk
vTuner	Internet Radio

Vendor	Description
_RemoteLocal	playlist on phone
CustomStream	Open Network Stream
RadioParadise	Radio Paradise
Prime	Amazon Music (A97)
CalmRadio	Calm Radio (A97)
SoundCloud	Sound Cloud (A97)
SoundMachine	Sound Machine (A97)

MORE INFORMATION

get media information

MCU+PINFGET

response message

AXX+PLY+INF{"type": "0", "ch": "0", "mode": "10", "loop": "0", "eq": "0", "status": "play", "curpos": "113756", "offset_pts": "113798", "totlen": "272000", "Title": "486561"}

This should be the most completed information for now playing status.

Field	Description
type	unkown
ch	unkown
mode	input source, refer to above PLM mode
loop	loop/shuffle mode, please refer to the above Playback Mode
eq	unkown
status	current playback status, might be play , stop , pause
curpos	milliseconds of elapsed
offset_pts	unkown
totlen	milliseconds of duration
Title	song title (hexed)
Artist	song artist (hexed)
Album	song album (hexed)
alarmflag	unkown
plicount	count of songs in playlist
plicurr	current index in playlist

Field	Description
vol	current volume
mute	mute status

Spotify Status

message from device when spotify started

```
AXX+SPY+001
```

The `SPY` message will have param `000` when spotify stopped, and `001` when spotify started.

Passthrough Function

The device is composed with 2 parts, the WiFi module and the base board. The basic functions described above will work with the WiFi module, so all devices have the same WiFi module (for example A31) will have same API.

And the passthrough function which is used for the client to communicate with the MCU on base board directly, and which have an extended function set which will work with the base board and might be different for various models.

Extended EQ function

sample command

```
MCU+PAS+EQGet&
```

response message

```
MCU+PAS+EQ:bass:05&MCU+PAS+EQ:treble:05&
```

Function	Description
EQGet	query current EQ, no param
EQSet:{type}:{value}	set EQ
EQ:{type}:{value}	current EQ

Param	Description
{type}	will be <code>treble</code> or <code>bass</code>
{value}	a number 0 ~ 10 which stands for -5 ~ 5 on APP (actually gain -10dB ~ +10 dB).

AP8064 platform

sample command

```
MCU+PAS+Rakoit:GetBoard&
```

response message

```
MCU+PAS+Rakoit:Board:A50C&
```

Models including the PRO v1/2, AMP v1/v2 and some A50/S50 are based on an old platform AP8064. Commands will be sent as param of `PAS` with an identifier: `Rakoit:`.

Command	Ver	Description
<code>GetBoard</code>	0	get current board ID
<code>GetCommit</code>	0	get git commit
<code>GetPrompt</code>	0	get current voice prompt settings
<code>SetPrompt:{pmt}</code>	0	enable/disable voice prompt, {pmt} should be 0 or 1
<code>GetAPIVer</code>	1	get API version
<code>SendKey:{key}</code>	1	send key
<code>MaxVolume:Get</code>	2	get max volume
<code>MaxVolume:{mxv}</code>	2	set max volume
<code>VB:INT:{vb_int}</code>	2	set intensity of virtual bass
<code>VB:ENH:{vb_enh}</code>	2	set enhance of virtual bass
<code>VB:SWI</code>	2	toggle virtual bass
<code>VB:Get</code>	2	get on/off state of virtual bass
<code>VB:{en}</code>	2	turn on/off virtual bass
<code>LED:{en}</code>	2	turn on/off LED

BP10XX platform

sample command

```
MCU+PAS+RAK0IT:VOL&
```

response message

```
MCU+PAS+RAK0IT:VOL:50&
```

Models including MINI V3, PRO v3, AMP v3 and newer are based on platform BP10XX. Command will be sent as param of `PAS` with an identifier: `RAK0IT:`. eg: to query system volume, you can send payload `MCU+PAS+RAK0IT:VOL&`, and will receive message with payload `MCU+PAS+RAK0IT:VOL:50&`. The extended passthrough function will have some abundant functions with the basic WiFi function, as it shares with the same command set with the UART interface.

For more command details, please refer to UART API.